

1. Research question and the controlled-comparison design

Central question. Across a shared set of GTFS edits, where does function calling win, where does free-form code generation win, and can a router learn to predict which to use?

Every method attempts every scenario. With the tasks held constant, the method becomes the only variable, and the result turns from two unrelated accuracy numbers into a boundary map of where each mechanism wins. This mirrors the controlled design of CodeAct (Wang et al., 2024), which pits executable-code actions against predefined tool calls on identical tasks.

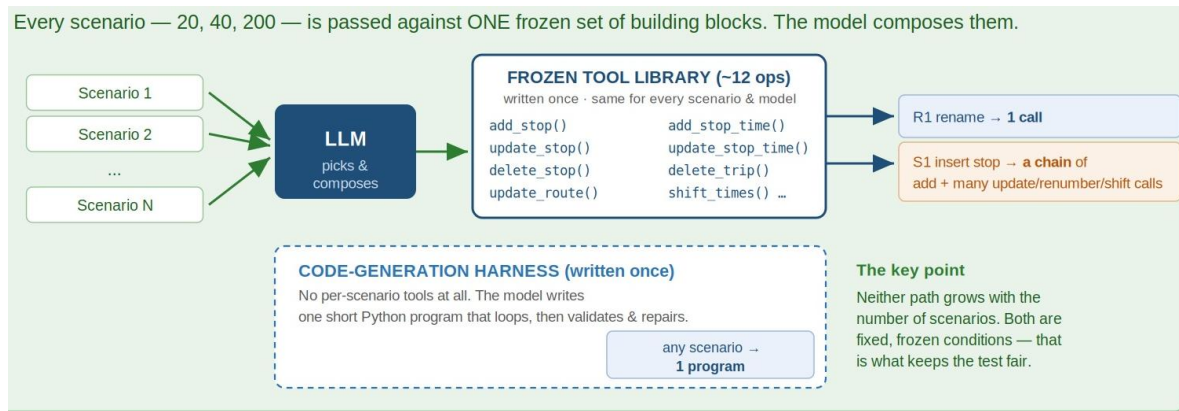
2. The three execution mechanisms

Each mechanism is a different way to turn an approved request into an actual edit of the feed:

1. **Function calling (FC).** The model fills parameters into a fixed library of editing operations (e.g. `rename_stop`, `set_route_color`). Clean and auditable for routine edits; awkward when an edit needs open-ended logic such as sequence renumbering plus time interpolation.
2. **Code generation + self-repair.** The model writes a small program that transforms the feed, runs it, and — if the validator fails — repairs and retries up to a budget. Powerful for structural edits; higher variance and cost.
3. **Chain-of-thought reasoning → FC or code-gen.** An explicit reasoning step precedes either executor, to test whether deliberation changes the win boundary.

2.1 The fixed function-calling library

The tools are designed around the types of edit, not the scenarios. Some general operation collection can be used to perform scenario based modification:

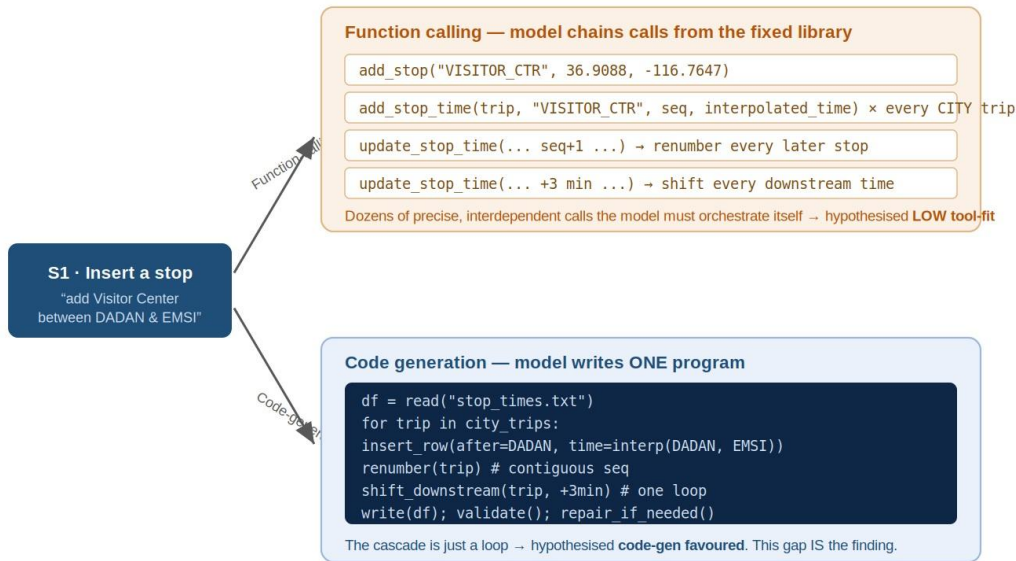


Fixed set of functions designed to solve the tasks related to GTFS modifications (sample)

Operation (frozen)	What it does
find_stop / find_trip / find_route	Look up an entity by name or attribute and return its id.
add_stop / update_stop / delete_stop	Create a stop, edit any of its fields, or remove it.
update_route / update_agency	Edit route or agency fields (name, colour, contact).
update_frequency	Edit a frequencies row (e.g. headway).
add_stop_time / update_stop_time / delete_stop_time	Insert, edit, or remove a single trip-stop row (time, sequence, stop).
delete_trip	Remove a trip and its stop_times.
shift_times(scope, offset)	Shift a set of times by a fixed offset (a deliberate design choice — see 5.5).

2.2 A scenario is solved by composition

Routine edits land on a single tool. Structural edits force the model to either compose a long, fragile, interdependent chain or fail because no tool cleanly fits. That gap is exactly what “tool-fit” names, and measuring it is the finding.



How the twenty scenarios map onto the one fixed library:

Scenario	How function calling handles it with the fixed library	Tool-fit
R1 / R2 / R5 rename, move, accessibility flag	one update_stop call	HIGH
R3 / R6 recolour, long name	one update_route call	HIGH
R4 / R7 agency, headway	one update_agency / update_frequency	HIGH
S1 insert a stop	add_stop + add_stop_time×(trips) + update_stop_time×(renumber every later stop) + update_stop_time×(shift every downstream time) — dozens of interdependent calls the model orchestrates and interpolates itself	LOW
S2 / S3 / S4 / S8 remove, reverse, split, merge	no single tool fits; long error-prone chains, or impossible to express cleanly	LOW
A1–A4 under-specified	the router asks before any tool is called	—

2.3 The validity rule

One fixed tool library, frozen before the scenarios are seen, identical for every model and every scenario, never tailored per task. The same goes for the code-gen harness. That is what keeps the comparison honest. It is directly analogous to how BFCL / Gorilla works: models are scored on selecting correctly from a fixed function set, and “knowing when no function applies” is itself a graded skill, which is precisely what the ambiguous A-group and the no-tool-fits structural cases test in our benchmark.

3. System architecture (shared by app and experiment)

In the chatbot, the router is the intellectual layer. A request arrives, the router classifies it once, and dispatches it to whichever mechanism fits. The app is the proof of concept that demonstrates this end-to-end; the model benchmarking runs the very same mechanisms and router offline over the fixed scenario set. One architecture, two modes of use.

① User request (+ feed metadata)

Plain-English instruction plus a snapshot of the loaded feed.



② ROUTER — the intellectual layer

Classifies the request once: tool-fit (high/med/low), ambiguous (yes/no), confidence (0–1). Decides the path.



③a Function calling tool-fit HIGH	③b Code-gen + repair tool-fit LOW	③c Clarify with user ambiguous
<i>medium → try FC, then fall back to code-gen + repair (spans both paths)</i>		



④ Edited feed

Candidate edit produced by the chosen mechanism.



⑤ Validator + answer-key grading

Official GTFS validator plus the scenario’s oracle: validity, correctness, integrity, damage, cost.

4. The router as the intellectual layer

The router looks at the request once and predicts a category, then sends it to the matching mechanism.

Input / output

- **Input:** request text + feed metadata.
- **Output:** { tool-fit: high | med | low, ambiguous: yes | no, confidence: 0–1 }.

Dispatch policy

- **tool-fit HIGH** → function calling.
- **tool-fit LOW** → code generation + repair.
- **tool-fit MEDIUM** → try function calling, then fall back to code-gen.
- **ambiguous** → ask the user to clarify before touching anything.

How the router is graded

Each scenario carries a human-assigned hypothesis about which mechanism should win. Several people (e.g. Zach, the author, and one more) label every scenario independently and then reconcile to a final label. The router's predicted tool-fit is scored against those labels:

- **tool-fit = high** → predict function calling wins.
- **tool-fit = low** → predict code generation wins.
- **tool-fit = medium** → predict a close call / contested.
- **ambiguous** → predict that clarifying the request matters most.

So the router has its own ground truth and its own accuracy number, layered on top of the head-to-head mechanism comparison.

	Scenario	Natural-language request	Expected diff (oracle answer key)	Validator / integrity check	Tool-fit
Group A — Routine parametric edits (predicted tool-fit HIGH → function calling)					
R1	Rename a stop	“Rename the stop ‘Stagecoach Hotel & Casino’ to ‘Stagecoach Casino’.”	stops.txt: 1 cell changed (stop_name). stop_id and all references unchanged.	Only stop_name changes; stop_id intact; every stop_times reference still resolves.	HIGH
R2	Move a stop’s coordinates	“Update the location of ‘Nye County Airport’ to (36.868446, −116.784582).”	stops.txt: stop_lat / stop_lon updated for that stop_id only.	lat ∈ [−90,90], lon ∈ [−180,180]; no other field or row altered.	HIGH
R3	Recolor a route	“Set the City route colour to 1E90FF and its text colour to FFFFFFFF.”	routes.txt: route_color / route_text_color for CITY.	Valid 6-digit hex; only that row’s colour fields change.	HIGH
R4	Update agency contact info	“Change the agency phone number to (775) 555-0143.”	agency.txt: agency_phone field.	Single field updated; agency_id, agency_url, timezone unchanged.	HIGH
R5	Flag a stop wheelchair-accessible	“Mark the ‘Bullfrog’ stop as wheelchair accessible.”	stops.txt: wheelchair_boarding = 1 for that stop (add column if absent).	Value ∈ {0,1,2}; only the target row affected.	HIGH
R6	Edit a route’s long name	“Rename route AB’s long name to ‘Airport – Bullfrog Express’.”	routes.txt: route_long_name for AB.	Single cell changed; route_id and short name preserved.	HIGH
R7	Change a frequency headway	“Change the AAMV midday headway from 30 to 20 minutes.”	frequencies.txt: headway_secs 1800 → 1200 on the matching row.	headway_secs > 0; start_time / end_time unchanged; one row touched.	HIGH
Group B — Structural / multi-file cascade edits (predicted tool-fit LOW → code generation + repair)					
S1	Insert a stop mid-route	“Add ‘Visitor Center’ at (36.9088, −116.7647) on the City route between Doing Ave / D Ave N and E Main St / S Irving St, adding 3 min of travel.”	stops.txt +1 row; for every CITY trip, insert a stop_times row between DADAN and EMSI, renumber stop_sequence for all later stops, interpolate the new time,	New stop_id referenced by ≥1 stop_time; stop_sequence contiguous & strictly increasing; stops before DADAN unchanged.	LOW

	Scenario	Natural-language request	Expected diff (oracle answer key)	Validator / integrity check	Tool-fit
			and shift downstream times +3 min.		
S2	Remove a mid-route stop	“Remove ‘Doing Ave / D Ave N’ from the City route and close the time gap.”	stop_times.txt: delete that stop’s rows across CITY trips, renumber sequences to stay contiguous, pull downstream times earlier.	Sequence stays 1..n contiguous; times monotonic; stop deleted from stops.txt only if unused elsewhere.	LOW
S3	Generate reverse-direction trips	“Create return-direction trips (direction_id = 1) for the City route by mirroring the outbound trips.”	trips.txt +N rows (direction_id 1); stop_times.txt with reversed sequences and recomputed times.	Each new trip mirrors stop order; times strictly increasing; valid shape/route references.	LOW
S4	Split a route into two	“Split route ‘City’ at ‘Nevada Square’ into ‘City North’ and ‘City South’.”	routes.txt +1 row; trips reassigned; stop_times partitioned at the split stop; new route_ids issued.	Every trip belongs to exactly one resulting route; no orphaned stop_times; both routes valid.	LOW
S5	Shift a service by a fixed offset	“Push every trip on weekday service FULLW 15 minutes later.”	stop_times.txt: +15 min on all arrival/departure times for trips whose service_id is FULLW.	Uniform shift; times remain monotonic; >24:00 GTFS convention respected where applicable.	LOW
S6	Interpolate missing stop times	“Fill the blank arrival/departure times on trip CITY1 by linear interpolation between timed stops.”	stop_times.txt: populate empty time fields on CITY1.	Filled times lie between bracketing timed stops; strictly increasing along the sequence.	LOW
S7	Extend a route with	“Extend AB past ‘Bullfrog’ with ‘Beatty Library’	stops.txt +2; stop_times.txt +2 rows per AB trip with new sequences and times;	Sequences contiguous; new stops referenced; appended	LOW

	Scenario	Natural-language request	Expected diff (oracle answer key)	Validator / integrity check	Tool-fit
	new terminal stops	(36.9047,−116.7558) and ‘Beatty Community Ctr’ (36.9120,−116.7501), 4 and 7 min later.”	shapes.txt extended if present.	times strictly increasing.	
S8	Merge two trips into one	“Merge trips AB1 and BFC1 into a single continuous through-trip.”	Combine stop_times under one trip_id, renumber the sequence, reconcile the join time, drop the now-empty trip.	Single contiguous sequence; monotonic times; no dangling trip_id references.	LOW
S9	Normalise stop_sequence values	“Renumber stop_sequence on all trips to run 1,2,3... with no gaps, preserving order.”	stop_times.txt: rewrite stop_sequence across many trips.	Each trip’s sequence is 1..n contiguous; relative stop order preserved everywhere.	LOW
Group C — Under-specified requests (predicted AMBIGUOUS → router asks before acting)					
A1	Vague frequency change	“Make the morning buses come more often.”	Under-specified: which route(s)? what counts as ‘morning’? how much more often?	Correct behaviour = ask before editing. No file is modified until route, time window, and target headway are supplied.	AMBIGUOUS
A2	Undefined ‘fix’ request	“The weekend schedule looks wrong — can you fix it?”	No defect identified, no target service, no correct intent stated.	Router must request the specific symptom and intended outcome; no edit performed on a guess.	AMBIGUOUS
A3	Locationless stop add	“Add a stop near downtown.”	No coordinates, no route, no insertion point.	Router must elicit coordinates, the target route, and the between-stops position before touching any file.	AMBIGUOUS

	Scenario	Natural-language request	Expected diff (oracle answer key)	Validator / integrity check	Tool-fit
A4	Unquantified speed-up	“This trip is too slow — speed it up.”	Which trip, which segment, and by how much are all missing.	Router must obtain trip_id, the affected segment, and the time delta before editing stop_times.	AMBIGUOUS

5. Which models are worth benchmarking

Because the router dispatches to function calling or code generation, the relevant evidence comes from two different public leaderboards. Snapshots below are current as of June 2026; both fields move monthly, so treat these as a starting shortlist to re-confirm at submission time.

5.1 Function-calling ability (Berkeley Function Calling Leaderboard, BFCL)

BFCL is the most-cited tool-use benchmark; its V4 release (updated April 2026) adds holistic agentic evaluation, while V3 remains the common cross-model reference. Recent top scorers are dominated by the Qwen family (Qwen3.7 Max led BFCL-V4 at ~75%), with GLM and Qwen variants strong on V3.

Important caveat for us. BFCL grades the structure of the emitted call via abstract-syntax-tree matching. Claude models often score lower there because they wrap calls conversationally even when the tool choice is correct — a grading artifact, not a real-world weakness. Our benchmark sidesteps this entirely: we grade the executed outcome (did the feed end up correct?), not the call’s surface syntax.

5.2 Code-generation ability (SWE-bench)

SWE-bench Verified is saturating (frontier models cluster at 80–95%) and was deprecated by OpenAI in Feb 2026 over contamination; SWE-bench Pro is the contamination-resistant successor and is now the better discriminator.

- **Frontier (Verified):** Anthropic’s Mythos-class models lead (~95%) but are currently suspended under a June 2026 US export-control directive; the practical picks are Claude Opus 4.8 (~88.6%) and GPT-5.5 (~88.7%).
- **Frontier (Pro):** Claude Opus 4.8 ~69.2% vendor-reported; GPT-5.4 ~59.1% on Scale’s standardized SEAL board.
- **Open-weight:** DeepSeek-V4 and MiniMax M3 sit near the top of the open pack (~80% Verified), with Qwen close behind — valuable for reproducibility and zero per-call cost.

Model	Tier	Why include it
Claude Opus 4.8	Frontier	Strong code-gen + agentic behaviour. Practical stand-in while Mythos-class models are suspended. (BFCL AST artifact is irrelevant to us — we grade outcomes.)
GPT-5.5 / GPT-5.4	Frontier	Strong on both axes; leads the standardized SWE-bench Pro board; competitive function calling.
Gemini 3.1 Pro	Frontier	Third major lab for breadth; strong coding and tool use.
Qwen3.7 Max	Open-weight	Leads BFCL-V4 function calling; reproducible and self-hostable — your function-calling anchor.
DeepSeek-V4	Open-weight	Top open code-gen; reproducibility anchor for the code-generation path.